51. BUS PASSENGER OCCUPANCY SIMULATION

1. AIM

To simulate passenger occupancy across multiple bus stops and determine whether the bus capacity is exceeded.

2. PROBLEM STATEMENT

Public transport buses need to track passenger counts dynamically across multiple stops to ensure passenger safety and prevent overcrowding. Write a Python program that accepts total bus capacity, number of stops, and boarding/alighting counts at each stop, and determines whether the bus becomes overloaded.

3. OBJECTIVES

- 1. Input total passenger capacity and total number of bus stops.
- 2. Use a loop to process passenger movement at each stop.
- 3. Update running total of passengers dynamically.
- 4. Compare passenger count against bus capacity and output status.

4. ASSUMPTIONS & REQUIREMENTS

Assumptions:

- Bus capacity and stop counts are positive integers.
- Initial passenger count before the first stop is 0.

Requirements:

- Python 3.x Interpreter
- IDLE / PyCharm / VS Code environment

5. ALGORITHM

Step 1: Read bus capacity and number of stops.

Step 2: Initialize passengers = 0.

Step 3: Loop for i from 1 to stops:

a. Read boarding count and leaving count.

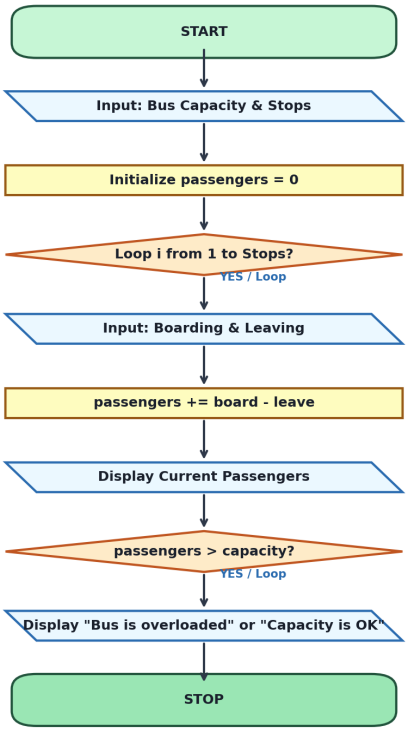
b. Update passengers = passengers + board - leave.

c. Display current passenger count.

d. If passengers > capacity, display 'Bus is overloaded', else display 'Capacity is OK'.

Step 4: Stop program.

6. PROGRAM FLOWCHART



7. PYTHON PROGRAM (bus.py)

```
capacity = int(input("Enter bus capacity: "))
stops = int(input("Enter number of stops: "))
passengers = 0

for i in range(1, stops + 1):
    board = int(input("Passengers boarding: "))
    leave = int(input("Passengers getting down: "))

    passengers = passengers + board - leave

    print("Passengers:", passengers)

    if passengers > capacity:
        print("Bus is overloaded")
    else:
        print("Capacity is OK")
```

8. EXPLANATION OF THE CODE

- **capacity:** Variable storing max seats available in the bus.
- **stops:** Total number of bus stops to iterate through.
- **passengers:** Accumulator variable tracking net passengers.
- **for loop:** Iterates through each bus stop in sequence.
- **if-else block:** Checks if current passengers exceed capacity and prints appropriate message.

9. TRACING TABLE

Iteration	Boarding	Leaving	Calculation	Passengers	Status Output
Initial	-	-	passengers = 0	0	-
Stop 1	45	20	0 + 45 - 20	25	Capacity is OK

10. TEST CASES

Test Case	Input	Expected Result	Actual Result
TC 1 (Normal)	Capacity: 50, Stops: 1 Board: 45, Leave: 20	Passengers: 25 Capacity is OK	Passengers: 25 Capacity is OK
TC 2 (Overload)	Capacity: 30, Stops: 1 Board: 40, Leave: 5	Passengers: 35 Bus is overloaded	Passengers: 35 Bus is overloaded
TC 3 (Boundary)	Capacity: 50, Stops: 1 Board: 50, Leave: 0	Passengers: 50 Capacity is OK	Passengers: 50 Capacity is OK

11. EXPECTED OUTPUT

```
Enter bus capacity: 50
Enter number of stops: 1
Passengers boarding: 45
Passengers getting down: 20
Passengers: 25
Capacity is OK
```

12. ACTUAL OUTPUT

```
Enter bus capacity: 50
Enter number of stops: 1
Passengers boarding: 45
Passengers getting down: 20
Passengers: 25
Capacity is OK
```

13. PROGRAM EXECUTION SCREENSHOT

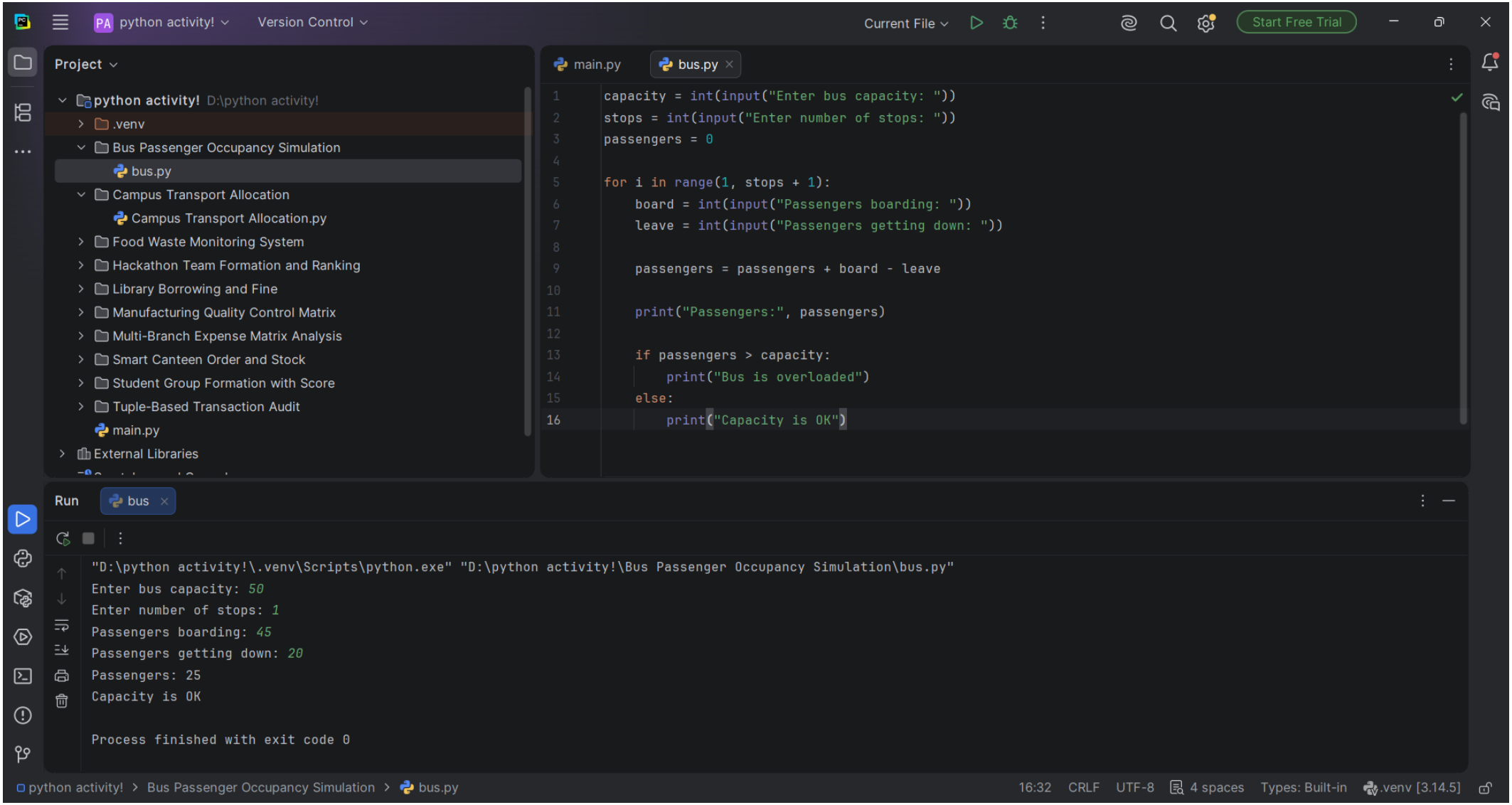
[Screenshot to Be Added]

14. RESULT / CONCLUSION

The Python program bus.py was executed successfully. It accurately simulates bus passenger occupancy across stops and detects capacity overload.

51 Bus Passenger Occupancy

Actual Input and Output – PyCharm Run Screenshot



Screenshot includes the program code and the console inputs/outputs.